1. 시스템 개요

본 시스템의 공식 명칭은 **ModuTwin**이며, 3D Gaussian Splatting(3DGS) 기반 모듈형 디지털 트윈 구축 플랫폼이다. 복도·로비 등 기준 공간(basemap)과 개별 방(module)을 각각 3DGS로 학습한 뒤, 문(door)을 매개로 공간을 접합하여 건물 전체의 디지털 트윈을 모듈 단위로 구축한다.

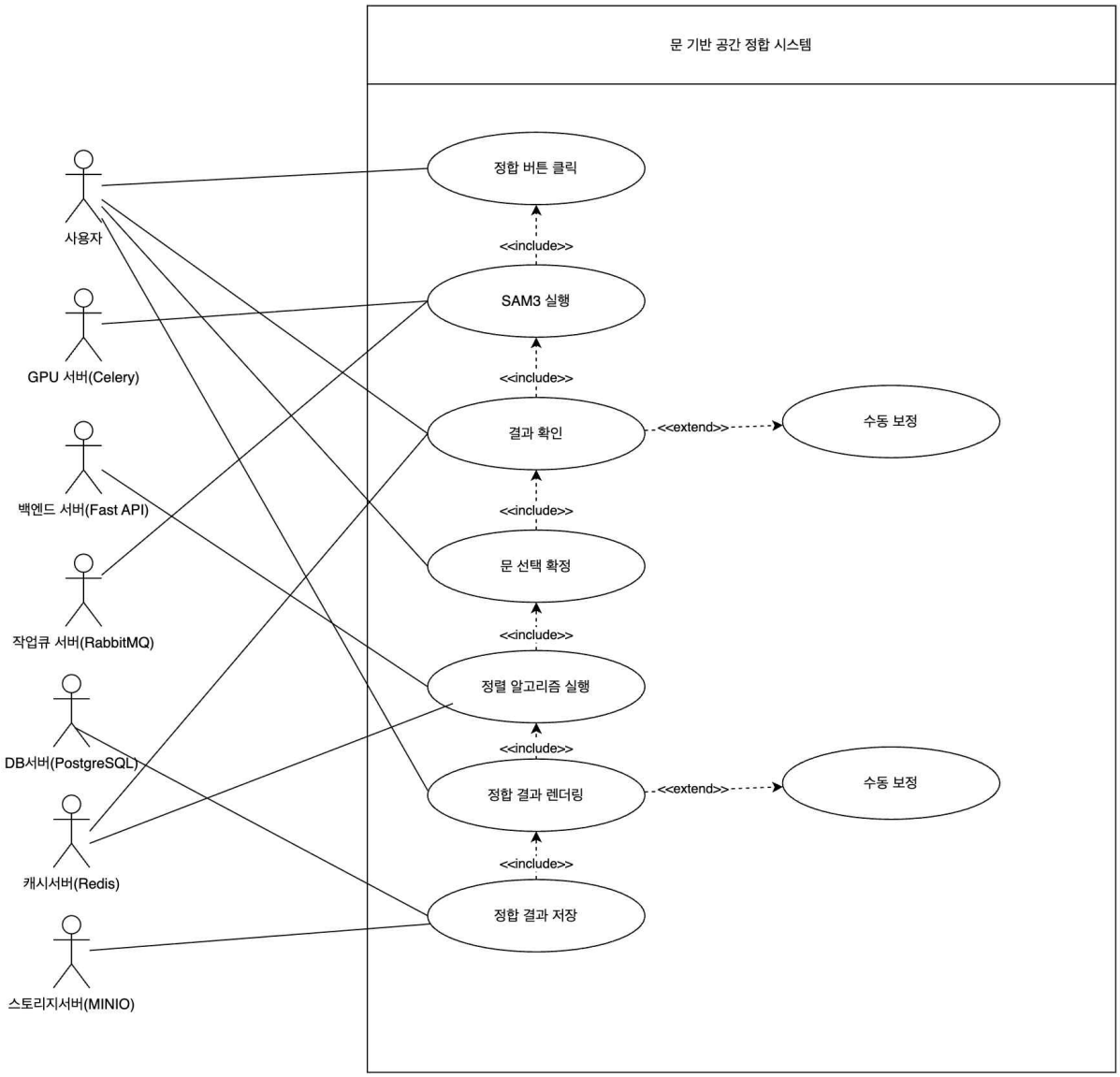
본 시스템은 웹 기반(React 프론트엔드 + FastAPI 백엔드)으로 동작하며, gsplat 학습 서버(RTX 4090), PostgreSQL DB, MinIO 오브젝트 스토리지, RabbitMQ 작업큐로 구성된다. 주요 사용자는 크라우드소싱 방식의 일반 참여자이며, 영상/이미지 업로드 → 3DGS 학습 → 문 기반 정합의 파이프라인을 통해 디지털 트윈을 구축한다.

2. Use Case

가. Actor 목록

Actor	구분	설명
사용자	시스템 사용자	로그인 후 영상/이미지를 업로드하고, 학습 완료된 모듈의 문 segmentation 확인·보정 및 정합 결과 미세 조정을 수행하는 사람
웹 서버 (FastAPI)	시스템	REST API 및 WebSocket을 제공하며, 인증(JWT), 태스크 발행, 상태 관리, presigned URL 생성을 수행
GPU 서버	시스템	RabbitMQ로부터 태스크를 수신하여 SAM3 segmentation 및 정합 알고리즘을 실행하는 워크스테이션 (RTX 4090)
스토리지 서버 (MinIO)	시스템	PLY, SOG, 인덱스 배열, 변환 행렬 JSON 등 파일을 저장·제공하는 오브젝트 스토리지
작업큐 (RabbitMQ)	시스템	웹 서버와 GPU 서버 사이의 비동기 태스크 발행·소비를 중개하는 메시지 브로커
DB 서버 (PostgreSQL)	시스템	업로드 메타데이터, 모듈-basemap 문 매핑 정보, 정합 상태 등을 저장·관리
캐시 서버 (Redis)	시스템	SAM3 segmentation 결과(door_indices), 변환 행렬 T, 정합 단계(phase) 등 정합 과정의 임시 상태를 저장·제공. 태스크 진행률 관리 및 WebSocket 온라인 여부 추적

나. 모듈 정합 (Module Alignment)



1) 개요

사용자가 학습 완료된 module의 "정합" 버튼을 클릭하면, 시스템이 SAM3 기반 자동 segmentation으로 module의 문 가우시안 입자를 추출하여 뷰어에 하이라이팅한다. 사용자는 결과를 확인하고, 필요 시 수동 편집 도구(바운딩박스, 페인팅)로 보정한 뒤 문 선택을 확정한다. 이후 시스템이 basemap의 대상 문의 GS 입자들과 module의 확정된 문의 GS 입자들을 이용해 강체 변환 행렬 T를 자동 계산하고, 정합된 결과를 뷰어에 표시한다.
사용자는 결과를 확인하고, 필요 시 회전/이동/스케일링 UI로 미세 조정한 뒤 최종 확정한다.

2) 사건흐름

- 기본흐름

<u>Phase A: 자동 Segmentation + 수동 보정</u>

1. 사용자가 대시보드에서 학습 완료된 module의 "정합" 버튼을 클릭한다.
2. 프론트엔드가 FastAPI에 정합 시작 요청을 전송한다.

   Request: POST /api/alignment/start

   Body: { module_id: str, door_id: str }
3. FastAPI가 RabbitMQ에 SAM3 segmentation 태스크를 발행한다.

   메시지: { task_type: "sam3_segmentation", module_id: str, ply_path: str, door_prompt: str }
4. FastAPI가 WebSocket을 통해 프론트엔드에 '세그멘테이션 진행중' 상태를 전송한다.
5. GPU 서버의 워커가 RabbitMQ에서 태스크를 수신한다.
6. GPU 서버가 MinIO에서 module PLY 파일을 다운로드한다.
7. GPU 서버가 SAM3 모델을 실행하여 문 영역의 가우시안 입자를 자동 추출한다.

   Input: PLY 파일 (가우시안 위치 좌표), 텍스트 프롬프트 (str)

   Output: 문 인덱스 배열 np.ndarray, dtype=np.intp, shape=(N,)
8. GPU 서버가 segmentation 결과(door_indices)를 Redis에 저장한다. FastAPI가 이를 읽어 WebSocket을 통해 프론트엔드에 전달한다.

메시지: { status: "success", door_indices: List[int], module_id: str }

9. 프론트엔드가 PlayCanvas 뷰어에서 module을 렌더링하고, 추출된 입자를 노란색 형광 하이라이트로 표시한다. 동시에 수동 편집 도구를 활성화한다.

10. 사용자가 결과를 확인한다.

    - 결과가 만족스러우면 → 11번으로 진행.

    - SAM3가 문을 여러 개 감지한 경우 → 사용자가 basemap 대상 문에 대응하는 문을 선택한다.

    - 결과가 불만족스러우면 → 수동 편집 도구로 보정한다:

        · 바운딩박스 도구: 3D 박스를 회전/이동/스케일링하여 영역 단위 선택/제거

        · 페인팅 도구: 다양한 각도에서 브러시로 입자 선택 (합집합/교집합/차집합 모드, 브러시 크기 조절)

11. 사용자가 "문 선택 확정" 버튼을 클릭한다.

<u>Phase B: 자동 정합 + 수동 미세 조정</u>

12. 프론트엔드가 FastAPI에 확정된 문 인덱스 배열과 함께 정합 실행 요청을 전송한다.

    Request: POST /api/alignment/execute

    Body: { module_id: str, door_id: str, module_door_indices: List[int] }

13. FastAPI가 RabbitMQ에 정합 태스크를 발행한다.

    메시지: { task_type: "door_alignment", module_id: str, door_id: str, module_ply_path: str, module_door_indices: List[int], basemap_ply_path: str, basemap_door_indices_path: str }

14. FastAPI가 WebSocket을 통해 프론트엔드에 '정합 진행중' 상태를 전송한다.

15. GPU 서버의 워커가 RabbitMQ에서 태스크를 수신한다.

16. GPU 서버가 MinIO에서 basemap PLY, basemap door_indices, module PLY를 다운로드한다.

17. GPU 서버가 양쪽 문의 GS 입자들에 대해 정합 알고리즘을 실행한다:

    a. RANSAC으로 문 평면 outlier 제거 (임계값 $\tau = k \cdot MAD / 0.6745$, 자동 계산)

b. Inlier GS 입자들에 SVD(PCA)를 적용하여 주축 추출 (a0: 세로, a1: 가로, a2: 법선)

c. GS 입자 밀도 비교로 법선 벡터가 방 바깥쪽을 향하도록 부호 결정

d. a0과 world_up (0,1,0)의 내적이 음수이면 a0의 부호를 반전하여 위를 향하도록 결정. 오른손 법칙으로 u축 결정

e. (u, v, n, c)로 4×4 프레임 행렬 F 구성

f. Inlier GS 입자들을 v축으로 투영 후 P1~P99 범위로 문 높이를 추정. s = h_basemap / h_module 비율을 u·v·n 세 축에 등방성 적용하여 스케일 행렬 S 계산

g. 최종 변환 행렬 T = F_basemap · S · F_module^{-1} 계산

18. GPU 서버가 정합 결과(변환 행렬 T, 스케일 비율)를 Redis에 저장한다. FastAPI가 이를 읽어 WebSocket을 통해 프론트엔드에 전달한다.

    메시지: { status: "success", transform_matrix: float64[4][4], scale: float64, module_id: str }

19. 프론트엔드가 PlayCanvas 뷰어에서 T를 module 전체에 실시간 적용하여 basemap과 합쳐진 결과를 렌더링한다. 동시에 미세 조정 UI(회전/이동/스케일링)를 활성화한다.

20. 사용자가 정합 결과를 확인한다.

    - 결과가 만족스러우면 → 21번으로 진행.
    - 결과가 불만족스러우면 → 회전/이동/스케일링 UI로 module 전체를 미세 조정한다.

21. 사용자가 "정합 확정" 버튼을 클릭한다.

<u>Phase C: 저장</u>

22. 프론트엔드가 FastAPI에 최종 변환 행렬을 전송한다.

    Request: POST /api/alignment/confirm

    Body: { module_id: str, door_id: str, transform_matrix: float64[4][4] }

    (자동 계산된 T(변환행렬) + 사용자 수동 미세 조정이 합쳐진 최종 행렬)

23. FastAPI가 MinIO에 변환 행렬을 JSON으로 저장한다.

    경로: buildings/{building_name}/{floor}/modules/{module_id}/transform.json

    내용: { transform_matrix: float64[4][4], scale: float64, door_id, module_id, confirmed_at }

24. FastAPI가 PostgreSQL에 정합 완료 상태를 업데이트한다.

    UPDATE modules SET alignment_status = 'completed', transform_path = '...' WHERE module_id = '...'

25. FastAPI가 WebSocket을 통해 프론트엔드에 '정합 완료' 확인을 전송한다.

- 대안흐름

<u>대안흐름 A: SAM3 자동 추출 실패</u>

7a. SAM3 모델이 문 영역을 인식하지 못하거나 빈 결과를 반환한 경우 → GPU 서버가 실패 결과를 Redis에 저장한다. FastAPI가 이를 읽어 WebSocket으로 오류를 전달 → 프론트엔드가 빈 선택 상태로 수동 편집기를 활성화한다. 사용자는 바운딩박스/페인팅 도구만으로 문 입자를 직접 선택한다.

<u>대안흐름 B: RabbitMQ 연결 실패</u>

3a/13a. RabbitMQ 연결 실패 시 → FastAPI가 500 오류를 반환하고, 프론트엔드가 재시도 버튼을 표시한다.

<u>대안흐름 C: MinIO 파일 다운로드 실패</u>

6a/16a. GPU 서버가 MinIO에서 파일 다운로드에 실패한 경우 → GPU 서버가 실패 결과를 Redis에 저장한다. FastAPI가 이를 읽어 WebSocket으로 오류 알림을 전송하고, 사용자에게 재시도를 안내한다.

<u>대안흐름 D: 스케일 이상치 감지</u>

17a. 계산된 스케일 비율 s가 비정상 범위(0.5 미만 또는 2.0 초과)인 경우 → GPU 서버가 경고 플래그를 포함하여 결과를 반환 → 프론트엔드에 경고 메시지 표시. 사용자는 segmentation 재확인 또는 미세 조정 UI로 대응한다.

<u>대안흐름 E: 사용자 취소</u>

10a. 사용자가 정합 과정을 취소 시 → 변경사항 없이 대시보드로 복귀한다.

3) 사전/사후조건

- 사전조건

1. module의 3DGS 학습이 완료되어 PLY 파일이 MinIO에 존재한다.
2. 업로드 시점에 해당 module이 부착될 basemap의 대상 문(door_id)이 지정되어 있다.
3. basemap의 해당 문이 이미 segmentation 완료되어, 문 인덱스 배열(door_indices)이 MinIO에 저장된 상태이다.
4. 사용자가 유효한 JWT Access Token을 보유하고 있다.

- 사후조건

1. MinIO에 transform.json이 저장된다.
2. PostgreSQL에 정합 완료 상태(alignment_status = 'completed')와 transform 경로가 기록된다.
3. 3D 뷰어(열람 모드)에서 해당 module이 basemap에 정합된 상태로 렌더링 가능하다.

4) 입출력 명세

Phase A (Segmentation) 입출력

구분	항목	자료형	설명
Input	module PLY 파일	binary (PLY format)	학습 완료된 module의 3DGS 가우시안 데이터. MinIO 경로로 참조.
Input	텍스트 프롬프트	str	SAM3에 전달할 문 설명 텍스트 (예: "하얀 문")
Input	module_id	str (UUID)	정합 대상 module의 고유 식별자
Input	door_id	str (UUID)	basemap에서 대상 문의 고유 식별자
Output	module 문 인덱스 배열	np.ndarray, dtype=np.intp, shape=(N,)	module PLY 전체 가우시안 중 문으로 선택된 입자들의 인덱스. 사용자 보정 후 확정된 최종값.

Phase B (정합 알고리즘) 입출력

구분	항목	자료형	설명
Input	basemap 문의 GS 입자들	np.ndarray, dtype=np.float64, shape=(M,3)	basemap PLY에서 door_indices로 추출한 가우시안 위치 좌표
Input	module 문의 GS 입자들	np.ndarray, dtype=np.float64, shape=(N,3)	module PLY에서 확정된 door_indices로 추출한 가우시안 위치 좌표
Input	basemap door_indices	np.ndarray, dtype=np.intp, shape=(M,)	basemap의 사전 segmentation된 문 인덱스
Input	module door_indices	np.ndarray, dtype=np.intp, shape=(N,)	Phase A에서 확정된 module 문 인덱스
Output	변환 행렬 T	np.ndarray, dtype=np.float64, shape=(4,4)	T = F_basemap · S · F_module^{-1}. module → basemap 좌표계 변환
Output	스케일 비율 s	float64	v축 P1~P99 퍼센타일 범위 기반 등방성 스케일 비율. s = h_basemap / h_module

Phase C (최종 저장) 입출력

구분	항목	자료형	설명
Input	최종 변환 행렬	float64[4][4] (JSON)	자동 T + 사용자 미세 조정이 합쳐진 최종 행렬
Output	transform.json	JSON file (MinIO)	{ transform_matrix, scale, door_id, module_id, confirmed_at }
Output	PostgreSQL 레코드	DB row	modules.alignment_status = 'completed', modules.transform_path 갱신

5) API 엔드포인트 명세

엔드포인트	Method / Path	Request Body	Response
정합 시작	POST /api/alignment/start	{ module_id: str, door_id: str }	{ status: str, task_id: str }
정합 실행	POST /api/alignment/execute	{ module_id: str, door_id: str, module_door_indices: List[int] }	{ status: str, task_id: str }
정합 확정	POST /api/alignment/confirm	{ module_id: str, door_id: str, transform_matrix: float64[4][4] }	{ status: str, transform_path: str }

6) RabbitMQ 메시지 명세

태스크	발행 메시지 (웹 서버 → GPU 서버)	완료 메시지 (GPU 서버 → Redis)
SAM3 Segmentation	{ task_type: "sam3_segmentation", module_id: str, ply_path: str, door_prompt: str }	{ status: "success" | "failure", door_indices: List[int], module_id: str }
Door Alignment	{ task_type: "door_alignment", module_id: str, door_id: str, module_ply_path: str, module_door_indices: List[int], basemap_ply_path: str, basemap_door_indices_path: str }	{ status: "success" | "failure", transform_matrix: float64[4][4], scale: float64, module_id: str }

7) MinIO 저장 경로 명세

경로 패턴	파일 형식	설명
users/{user_id}/{building_name}/web_input/	PLY (binary)	학습 완료된 module PLY 파일
basemaps/{basemap_id}/doors/{door_id}/door_indices.npy	NumPy (.npy)	basemap 문의 사전 segmentation 인덱스 배열
buildings/{building_name}/{floor}/modules/{module_id}/transform.json	JSON	최종 변환 행렬 및 메타데이터

8) 정합 알고리즘 세부 단계 참조

#	단계	목적
1	RANSAC	손잡이 등 비평면 outlier 제거 (임계값 $\tau = k \cdot MAD / 0.6745$)
2	SVD(PCA)	문의 GS 입자들에서 주축 추출 (a0: 세로, a1: 가로, a2: 법선)
3	밀도 비교	법선 벡터가 방 바깥쪽을 향하도록 부호 결정
4	세로축 부호 결정	a0과 world_up (0,1,0)의 내적이 음수이면 a0의 부호를 반전하여 위를 향하도록 결정. 오른손 법칙으로 u축 결정
5	스케일 보정	Inlier GS 입자들을 v축으로 투영 후 P1~P99 범위로 문 높이를 추정. s = h_basemap / h_module 비율을 u·v·n 세 축에 등방성 적용
6	프레임 합성	$T = F_basemap \cdot S \cdot F_module^{-1}$

9) 관련 화면

화면 1: Base Map 문 선택 화면

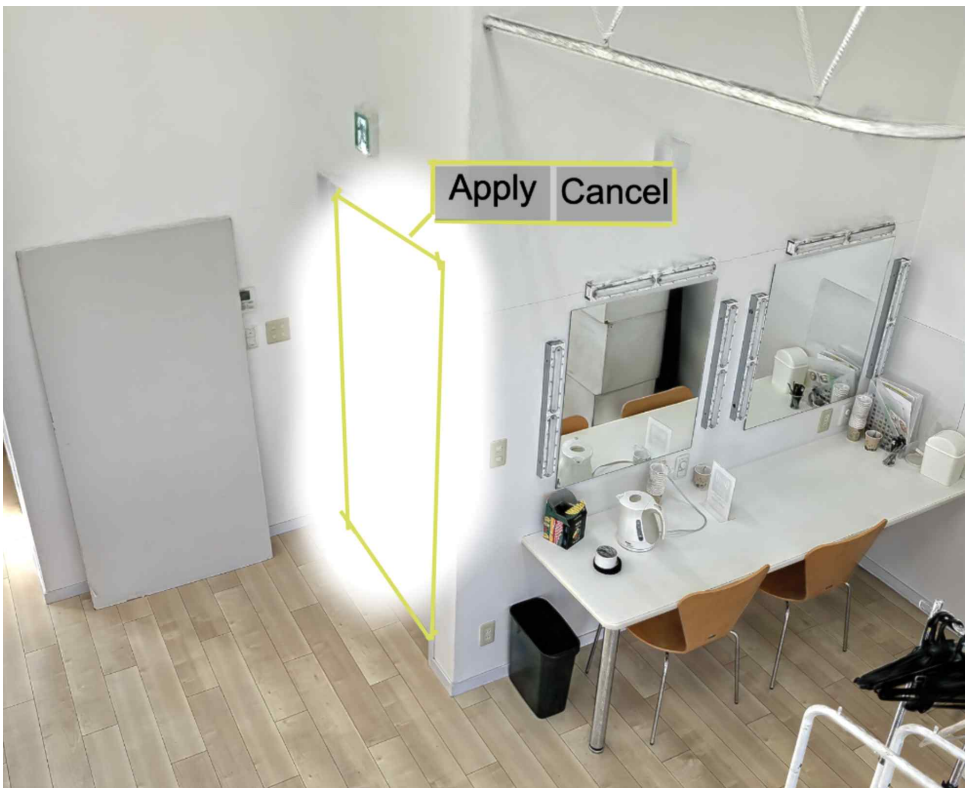
3D 뷰어에 Base Map이 로드된 상태에서, 참여자가 정합할 문을 클릭하여 선택하는 화면이다. 선택된 문은 하이라이트로 표시된다.

화면 2: 문 영역 지정 화면

Segmentation 결과가 불만족스러운 경우, 참여자가 바운딩 박스를 이동 및 스케일링하여 문 영역에 해당하는 가우시안 입자를 선택한다. 브러쉬를 활용한 편집 방법도 있다.

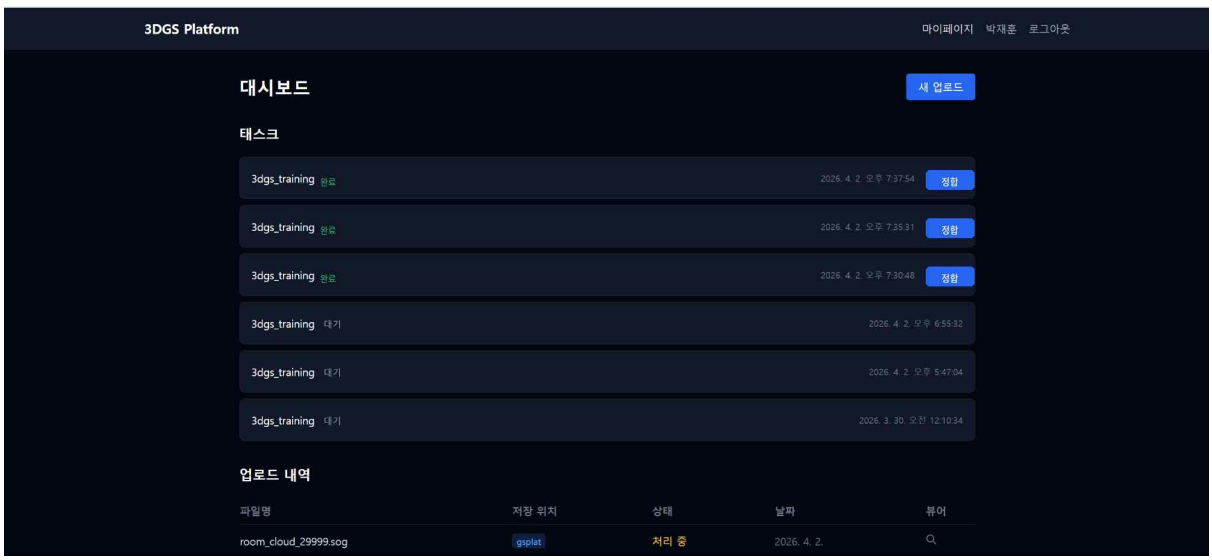
화면 3: 정합 결과 미리보기 및 Apply/Cancel 화면

시스템이 정합을 수행한 후, Base Map과 모듈이 결합된 상태를 3D 뷰어에서 미리보기로 표시한다. 화면에 Apply 버튼과 Cancel 버튼이 제공되며, 참여자는 정합 결과를 시각적으로 확인한 후 승인 또는 거부를 결정한다.



화면 4: 대시보드 업로드 화면

내가 업로드한 영상들과 학습진행률을 확인할 수 있는 대시보드이다. 정합 버튼을 누르면 두 공간 정합 코드가 실행된다.



화면 5: 지도 및 접속할 건물 선택

사용자가 지도에서 찾고자 하는 건물을 검색한다. 각 건물을 클릭했을 때 층에 대한 정보가 나타난다.

case A) 원하는 층을 클릭하면 해당 층의 정보가 제공된다.

case B) 층을 선택하면 해당 층을 볼 수 있는 뷰어로 이동한다.